SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1417

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 3 **Total Marks: 50**

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I K.Manashvitha(1925260260) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.Manashvitha

Introduction

This report answers the three questions in the Error Analysis Task for Compiler Design. It discusses token recognition, state transitions, longest-match rules, LEX/FLEX specification, token-stream validation, symbol-table interaction, intermediate code, and dependencies between compiler phases.

Q1. Analyze Token Recognition and Lexical State Transitions

(a) Lexeme and Token Analysis

Input program:

```
int count = 25;

while (count >= 10) {

    count = count - 5;

}

result = count != 0;
```

Lexemes and token classes, in order:

Lexeme	Token class
int	Keyword
count	Identifier
=	Assignment operator
25	Integer constant
;	Separator
while	Keyword
(	Separator
count	Identifier
>=	Relational operator
10	Integer constant
)	Separator
{	Separator
count	Identifier

=	Assignment operator
count	Identifier
-	Arithmetic operator
5	Integer constant
;	Separator
}	Separator
result	Identifier
=	Assignment operator
count	Identifier
!=	Relational operator
0	Integer constant
;	Separator

Complete token stream:

KW(int), ID(count), ASSIGN(=), INT(25), SEP(;), KW(while), SEP((), ID(count), REL(>=), INT(10), SEP()), SEP({),

ID(count), ASSIGN(=), ID(count), ARITH(-), INT(5), SEP(;), SEP(}), ID(result), ASSIGN(=), ID(count), REL(!=), INT(0), SEP(;)

The scanner reads from left to right and skips whitespace. A token ends when the next character cannot belong to its pattern. The longest-match rule selects the longest valid lexeme; if two rules match the same length, rule priority resolves the tie.

(b) State-Based Analysis

- Identifier DFA idea: from the start state, a letter enters the identifier state. Letters and digits remain in that state. A character outside the identifier pattern ends the token.
- Integer DFA idea: a digit enters the integer state; further digits remain in it. The first non-digit ends the integer token.
- For >=, read > and check the next character. If it is =, emit GE (>=); otherwise emit GT (>) and process the next character separately.
- For !=, read ! and check the next character. If it is =, emit NE (!=). A standalone ! is a lexical error unless the language defines it as a token.
- Whitespace is ignored. Separators such as parentheses, braces, and semicolons are emitted as tokens. At end of input, any pending identifier or integer is finalized and then EOF is returned.

(c) Longest-Match and Rule Priority

The operator >= must be recognized as one token because it is a two-character relational operator. Splitting it into > and = would produce an incorrect token stream. The same principle applies to !=. LEX/FLEX selects the rule matching the most characters; when matches have equal length, the earlier rule wins. Keyword rules may precede the identifier rule, or the scanner can match an identifier and then check a keyword table.

(d) Analytical Extension

1. Revised regular expression for identifiers with underscores: [A-Za-z_][A-Za-z0-9_]*

2. Add do as a keyword. Match the complete identifier and compare it with the keyword list: do is KEYWORD(DO), while double is ID(double). Longest-match prevents double from being split into do plus uble.

3. Suggested tests:

int count_1 = 42; while (count_1 >= 10) { count_1 = count_1 - 1; }

do { total = total + 1; } while (total != 0);

These tests check underscores, keywords, integer constants, and multi-character relational operators. The second assumes the language has been extended to support do.

Q2. Design a LEX-Based Token Recognizer for a Mini Expression Language

(a) Token Specification

Suitable regular expressions:

Identifier: [A-Za-z_][A-Za-z0-9_]*

Floating-point constant: [0-9]+\.[0-9]+

Integer constant: [0-9]+

Whitespace: [\t\r\n]+

Comparison operators: <=|>=|==|!=|<|>

Assignment operator: =

Separators: [(){};,]

Single-line comment: //[^\n]*

Arithmetic operators: [+*/%-]

Keywords: let, print, if (exact whole-word matches)

Keywords overlap with identifiers because both contain letters. Exact keyword rules and longest-match behavior ensure that print is a keyword while printer is one complete identifier. Integer and floating-point patterns both begin with digits, so the floating-point rule should be specified before the integer rule; longest-match selects 12.5 as one floating-point token.

(b) LEX/FLEX Program Design

The following FLEX specification prints each token type and lexeme. Whitespace and comments are ignored.

```
%{

#include <stdio.h>

%}

%option noyywrap

ID      [A-Za-z_][A-Za-z0-9_]*

FLOAT   [0-9]+\.[0-9]+

INTEGER [0-9]+

WS      [ \t\r\n]+

%%

"let"       { printf("KEYWORD: %s\n", yytext); }

"print"     { printf("KEYWORD: %s\n", yytext); }

"if"        { printf("KEYWORD: %s\n", yytext); }

{FLOAT}     { printf("FLOAT: %s\n", yytext); }
```

```
{INTEGER}    { printf("INTEGER: %s\n", yytext); }

{ID}         { printf("IDENTIFIER: %s\n", yytext); }

"<="|">="|"=="|"!="|"<"|">"  { printf("COMPARISON: %s\n", yytext); }

"="          { printf("ASSIGNMENT: %s\n", yytext); }

[+*/%-]      { printf("ARITHMETIC: %s\n", yytext); }

[(){};,]     { printf("SEPARATOR: %s\n", yytext); }

\\/[^\n]*  { /* ignore single-line comment */ }

{WS}         { /* ignore whitespace */ }

.            { printf("LEXICAL_ERROR: %s\n", yytext); }

%%

int main(void) { yylex(); return 0; }
```

Save as mini.l and compile with Flex and a C compiler. The comment rule is placed before the generic error rule. Longest-match handles multi-character operators; equal-length matches use rule order.

(c) Token Stream Validation

Input:

```
let total = 125;

let rate = 12.5;

print(total + rate * 2);

if (total >= 100) {

   // display the calculated value

   print(total);

}
```

Expected token stream (whitespace and comment omitted):

KEYWORD(let), IDENTIFIER(total), ASSIGNMENT(=), INTEGER(125), SEPARATOR(;)

KEYWORD(let), IDENTIFIER(rate), ASSIGNMENT(=), FLOAT(12.5), SEPARATOR(;)

KEYWORD(print), SEP((), ID(total), ARITH(+), ID(rate), ARITH(*), INT(2), SEP()), SEP(;)

KEYWORD(if), SEP((), ID(total), COMP(>=), INT(100), SEP()), SEP({)

KEYWORD(print), SEP((), ID(total), SEP()), SEP(;), SEP(})

The line comment is recognized and discarded. print matches a keyword exactly. printer is longer than print and is recognized as IDENTIFIER(printer), not as print followed by er.

(d) Testing and Evaluation

Test 1 — floating-point value:

let price = 7.25;

Expected: KEYWORD(let), ID(price), ASSIGN(=), FLOAT(7.25), SEP(;)

Test 2 — multi-character relational operator:

if (a != 10) print(a);

Expected: KEYWORD(if), SEP((), ID(a), COMP(!=), INT(10), SEP()), KEYWORD(print), SEP((), ID(a), SEP()), SEP(;)

Test 3 — identifier containing underscores:

let _item_count2 = 3;

Expected: KEYWORD(let), ID(_item_count2), ASSIGN(=), INT(3), SEP(;)

These tests verify decimal-number recognition, longest-match behavior for !=, and identifiers with leading and embedded underscores.

Q3. Analyze the Interaction Between Lexical Analysis and Subsequent Compiler Phases

(a) Lexical Analysis

Input:

int calculate(int a, int b) {

```
    int result;
    result = a + b * 10;
    return result;
}
```

Lexemes and classes: int—keyword; calculate—identifier; parentheses, commas, braces and semicolons—separators; a, b and result—identifiers; =—assignment operator; + and *—arithmetic operators; 10—integer constant; return—keyword.

Token stream:

KW(int), ID(calculate), SEP((), KW(int), ID(a), SEP(,), KW(int), ID(b), SEP()), SEP({),

KW(int), ID(result), SEP(;), ID(result), ASSIGN(=), ID(a), ARITH(+), ID(b), ARITH(*), INT(10), SEP(;),

KW(return), ID(result), SEP(;), SEP(})

Whitespace separates lexemes but is not normally emitted as a token. Punctuation is recognized as individual separator tokens.

(b) Symbol Table and Syntax Analysis

The identifiers calculate, a, b and result contribute entries to the symbol table. Useful information includes identifier name, kind (function, parameter, or local variable), data type, scope, parameter position, and possibly storage location.

Name	Kind	Type	Scope/details
calculate	Function	int return type	Global; has parameters a and b
a	Parameter	int	Local to calculate; parameter 1
b	Parameter	int	Local to calculate; parameter 2
result	Local variable	int	Local to calculate

The syntax analyzer checks whether tokens fit grammar rules for a function definition, parameter list, local declaration, assignment, and return statement. The lexer provides token classes and lexemes; the parser determines grammatical structure.

(c) Intermediate Representation Analysis

For result = a + b * 10, multiplication has higher precedence than addition. A suitable three-address-code representation is:

t1 = b * 10

t2 = a + t1

result = t2

Temporary variables t1 and t2 store intermediate values. Multiplication is represented first to preserve operator precedence.

(d) Phase Dependency Analysis

Lexical analysis supplies tokens and lexemes to syntax analysis. Syntax analysis checks grammatical structure and produces a parse tree or abstract syntax tree. Semantic analysis then checks issues such as type compatibility and whether identifiers are declared.

If * is incorrectly classified as an identifier, the parser will not receive the multiplication-operator token it expects. The expression may be rejected or misinterpreted, and later semantic analysis and code generation may be blocked. An incorrect token can therefore cause cascading errors across compiler phases.

(e) Advanced Compiler Perspective

- Improvement 1: Create unit tests for every token type and boundary case, including keywords, numbers, identifiers and multi-character operators.
- Improvement 2: Add clear lexical-error handling and track line and column positions.

Line and column information lets the compiler identify the exact location of an error, making debugging faster.

Additional test: if (a >= b) a = a + 1; Expected recognition includes GE(>=), ASSIGN(=), and PLUS(+). This checks that >= is not split into > and =.

Conclusion

Lexical analysis is the first compiler phase and directly affects later phases. Correct regular expressions, longest-match behavior, keyword recognition, state transitions, and useful error messages produce a reliable token stream. A well-tested LEX/FLEX scanner supports syntax analysis, symbol-table construction, and intermediate-code generation.

RUBRICS FOR EVALUATION					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Error Identification	20%	Accurately identifies all errors with clear understanding of issues.	Identifies most errors with minor omissions.	Identifies limited errors; some are missed.	Fails to identify key errors.
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning.	Explains causes with moderate clarity.	Partial explanation; weak reasoning.	Unable to identify causes of errors.
Correction & Justification	25%	Provides correct solutions with strong justification and logic.	Provides correct corrections with some justification.	Corrections are partially correct or weakly justified.	Incorrect or no corrections provided.
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve problems.	Applies concepts with minor errors.	Limited application; requires guidance.	Unable to apply concepts.
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically.	Generally clear with minor issues.	Lacks clarity or proper structure.	Poor presentation and difficult to understand.